

Using the Observer Pattern

Software Design & Programming

Object-Oriented Programming II

Joseph DiBiasi

University of Denver College of Professional Studies

5/2/2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Bobbie Kite, MLS

Introduction

The newest feature being incorporated into our Parking System application is the parking observer class. This new class is central to a rework of the original park and leave methods located in the transaction manager. The goal of this redesign is to implement the observer pattern in how we are creating our parking charges. We hope to decouple our event monitoring system to provide more flexibility as our system grows ever larger in both size and complexity.

Requirements

There were some initial development requirements that we wanted to adhere to. There should be a central blueprint interface class for implementing our observers. Our parking lots should be able to register one or more of these observers as needed and also remove them if necessary. Our existing park and leave methods needed to be reworked to incorporate this new observer notification structure. These requirements gave us a starting point on how we wanted to design our observer system.

High-Level Design

The parking observer class was designed to handle the creation of the observers and their registration with the parking office. This class also implemented an update method to notify the transaction manager of the parking events. This update method was required by a default parking action interface which was implemented by the parking observer to ensure consistency. When the observer was notified of an update, the transaction manager would then

invoke its park method. The parking event was an object in and of itself, containing information needed to create parking charges, and is passed directly into the park method. This allowed the same event to be used regardless of if a car was entering or leaving the parking lot.

The parking lots were given the responsibility of maintaining these observers; adding or removing them as required. Parking lots also gained new functionality in the form of park and leave methods; these methods were designed to trigger when cars entered or exited the lot. Once triggered, each method would then notify its observers of the event; these observers would then in turn notify the transaction manager to run the appropriate parking logic for tracking the parking charges incurred. This closely follows the observer pattern we desired; the parking lot class is the subject, and it is notifying the transaction manager class indirectly through the use of the parking observer.



Figure 1. Observer Pattern UML Diagram Regarding Parking Charges

Implementation

The actual implementation of this process relied on a well-known strategy in our Parking System application. Enums relating to entry and exit were used by the parking lot class to clearly communicate in its messages what type of parking event was just sent to the transaction manager. More information than this was required to successfully process a parking charge via the transaction manager, so our implementation used another familiar process and created a parking event class. This class was designed to transfer data and not manipulate it so while there are getter classes provided, there are no setters; each variable is declared final upon creation. Since the transaction manager had previous logic pertaining to park and leave methods, we already knew what data we needed the parking lot to pass in and wanted to ensure it would not be changed before reaching the transaction manager.

The fact that our Parking System application already had the park and leave functionality built out in the transaction manager was a definite boost to our development of the parking observer, but it was not without its challenges. Since the parking lot is decoupled from the transaction manager and communicates through updates in the parking observer, it was confusing at times navigating through the whole process to confirm it was correct. The integrated development environment (IDE) we were using to aid in development does allow one to click through the links, which helped; we also tried using the same naming pattern in each class for the park and leave functionality. Similar and consistent names might have served us better, as this also resulted in some confusion about where each aspect of the functionality should live. Searching by method name would turn up every instance, not just the instance we wanted to look at. Future cleanup work may revolve around renaming these methods to have a

hint about their location as well as their purpose. In the end, we were able to connect all of the pieces together despite these difficulties thanks to diligence and utilizing the provided version management tools within our IDE to track our changes.

Summary

Creating a decoupled notification system to replace our original parking charge tracking methods was no easy feat, but it was a worthwhile endeavor. Inputting an observer system that is separate from the source of events and the logic of what they trigger let us build a structure that is flexible and can adapt to the growing needs of our application. From these humble beginnings, our parking lots could potentially expand their observer functionality to cover a wide array of different features, such as tracking user's car location or providing a warning when parking lots are approaching capacity.